Precision Routing for Efficient Robot Policy Deployment on Edge Devices

CS 2420 Project Proposal

Brianna Grissom, Hang Le, Shresth Verma, Lester Xue

October 17, 2026

1 Introduction

Robot policies are increasingly built on top of large vision-language models (VLMs). Vision-language-action (VLA) models such as RT-2, OpenVLA, and π_0 fine-tune a pretrained VLM backbone to map camera images and a natural-language instruction directly to robot actions. Inheriting internet-scale visual and semantic knowledge allows a single generalist policy to follow open-ended instructions, recognize novel objects and scenes, and transfer across tasks and embodiments far better than policies trained from scratch on robot data alone.

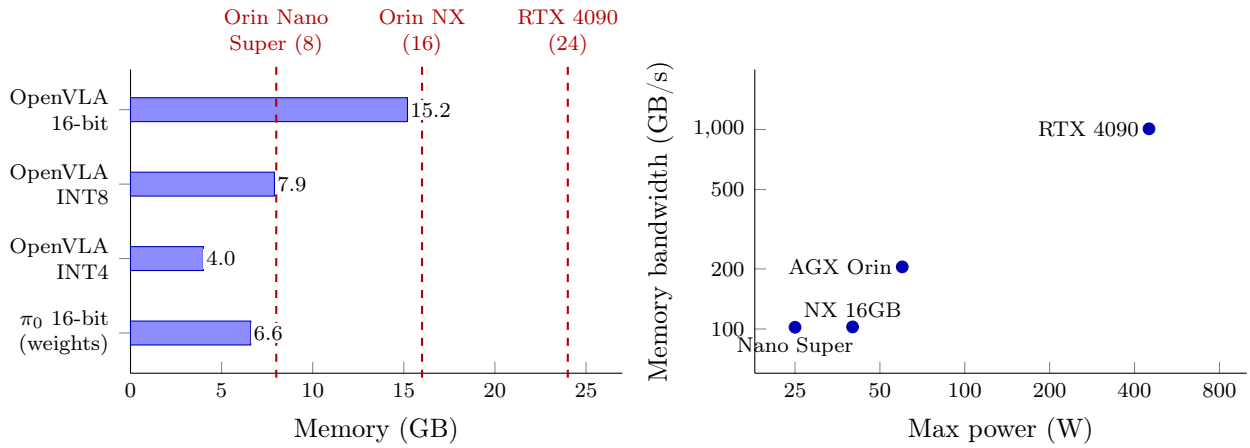
This capability comes at a substantial computational cost. VLA policies impose memory, bandwidth, and power demands that exceed what is available onboard most robots, yet closed-loop control requires low and predictable latency that offloading to a remote server cannot guarantee. OpenVLA requires approximately 15 GB in a 16-bit configuration, nearly double the entire 8 GB shared memory of a Jetson Orin Nano Super. Even the smaller π_0 contains approximately 3.3 billion parameters, requiring about 6.6 GB for its 16-bit weights alone, before accounting for inference buffers, perception, and control software. Figure 1 summarizes this gap between policy requirements and edge hardware.

Prior work has shown that quantization is an effective way to close this gap. On a Jetson AGX Orin, QAIL reports that INT4 weight quantization reduces OpenVLA's memory footprint from 15.2 to 4.0 GB and its inference latency from 955 to 375 ms, an approximately 2.5× However, quantization also reduces task accuracy. In the same evaluation, 4-bit AWQ lowers OpenVLA's average LIBERO success from 74.0% to 70.8%, and LIBERO-Long success from 50.8% to 47.2%. This leads to the central question of our project:

> *How can robot policies be deployed efficiently on compute-limited edge devices while achieving both a high success rate and low latency?*

Specifically, we propose learning a *router* that, at each control step, switches among quantized versions of the same policy based on the complexity of the current state. Our hypothesis is that *not all states require equal precision*: moving the arm through free space tolerates coarse actions, whereas grasping, insertion, and other contact-rich phases demand precise ones. Routing easy states to low-precision models and reserving high precision for critical states should therefore reduce average latency and energy while preserving the success rate of the full-precision policy.

Previous work on dynamic precision for robot policies relies on heuristic routers. DyQ-VLA switches activation precision using a hand-designed motion score with fixed thresholds, and MotionVQ switches between coarse and fine weights based on a fixed motion threshold. Such heuristics



(a) Policy memory vs. device memory.

(b) Bandwidth vs. power budget.

Figure 1: The deployment gap. (a) Memory needed by OpenVLA at different precisions and by π_0's 16-bit weights, against the memory of edge devices (dashed). Jetson memory is shared by CPU and GPU. (b) Edge modules trade memory bandwidth, which bounds batch-size-one decoding, for a lower power budget. Sources: OpenVLA, π_0, QAIL, and NVIDIA specifications; power figures are specified maxima, not measured inference power.

must be tuned by hand and use motion as a proxy for how much precision a state actually needs. Instead, we propose a lightweight router learned offline. We label states visited by the policy with the estimated loss in performance from acting at each precision, train a small classifier on cheap features (proprioception, action history, and previous precision choices) to predict the cheapest acceptable precision, and refine it with DAgger-style data aggregation so that training states match those the routed policy visits. Because the router operates on features that are already available, its overhead is negligible relative to the savings from reduced precision.

2 Problem Formulation

We model robot manipulation as a finite-horizon decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel P, reward function r, initial-state distribution ρ_0, and horizon T. The input to the policy at time t, which we write s_t, consists of the camera image, proprioception, and the language instruction. An episode $\tau = (s_0, a_0, \ldots, s_T)$ receives return $R(\tau) = \sum_{t=0}^{T-1} r(s_t, a_t)$. The reward may be sparse, such as a binary task-success signal at the end of the episode, or denser, such as progress toward the goal. We do not assume access to a learned value function, since VLA policies are trained by imitation rather than reinforcement learning.

Let π_1 be a pretrained full-precision VLA policy. Post-training quantization of π_1 yields a family of frozen policies $\{\pi_k\}_{k=1}^{K}$ at bit-widths $\mathcal{B} = \{b_1 > b_2 > \cdots > b_K\}$, with π_1 serving as the reference. Each policy has a per-call inference cost $c_k > 0$ (latency or energy measured on the target hardware), with $c_1 > c_2 > \cdots > c_K$.

A router $g_\phi(k \mid z_t)$ selects which policy to query at each step from a cheap feature vector $z_t = f(s_t, a_{t-1}, k_{t-1})$, such as proprioception, recent actions, and the previous precision choice. The routed policy π_ϕ acts as

$$k_t \sim g_\phi(\cdot \mid z_t), \qquad a_t \sim \pi_{k_t}(\cdot \mid s_t). \tag{1}$$

Our goal is to maximize expected return while minimizing compute:

$$\max_{\phi} \; J(\pi_\phi) - \lambda\, C(\pi_\phi), \qquad J(\pi_\phi) = \mathbb{E}_{\tau\sim\pi_\phi}\big[R(\tau)\big], \qquad C(\pi_\phi) = \mathbb{E}_{\tau\sim\pi_\phi}\Big[\sum_{t=0}^{T-1} c_{k_t}\Big], \tag{2}$$

where $\lambda \geq 0$ trades off return against compute.

2.1 Estimating the Precision Gap

Optimizing Eq. (2) directly would require knowing how a single low-precision decision changes the expected return, which is unavailable without a critic. Instead, we treat the full-precision policy as the reference behavior: a state tolerates low precision if the quantized policy behaves like π_1 there. We therefore define the *precision gap* $\Delta_k(s)$ of policy k at state s as how much π_k diverges from π_1 starting from s, with $\Delta_1(s) = 0$. We consider two estimators, in increasing order of cost and fidelity.

Myopic action divergence. Running all K policies on the same input s gives their action distributions directly. For OpenVLA, an action is a sequence of $J = 7$ discrete tokens $a = (a^1, \ldots, a^J)$, so by the chain rule the KL divergence decomposes exactly over tokens:

$$\hat{\Delta}_k^{\mathrm{KL}}(s) = \sum_{j=1}^{J} \mathbb{E}_{a^{<j}\sim\pi_1}\Big[D_{\mathrm{KL}}\big(\pi_1(a^j \mid s, a^{<j}) \,\|\, \pi_k(a^j \mid s, a^{<j})\big)\Big], \tag{3}$$

which we evaluate by teacher-forcing both models on the tokens decoded by π_1. For policies with continuous actions such as π_0, we use the expected squared distance between actions, $\hat{\Delta}_k^{\ell_2}(s) = \mathbb{E}\big[\|a^{(1)} - a^{(k)}\|_2^2\big]$, with shared sampling noise. This estimator costs one forward pass per policy but ignores whether a deviation actually affects the outcome: the policy can recover from a small deviation in free space but not from a missed grasp.

Multi-step rollout divergence. To capture whether a deviation compounds or is corrected, we reset the simulator to s, execute one action from π_k, and then follow π_1 for H steps, comparing against a rollout that follows π_1 throughout with the same random seed:

$$\hat{\Delta}_k^{H}(s) = \frac{1}{M}\sum_{m=1}^{M} d\big(x_{m,H}^{(k)}(s),\, x_{m,H}^{(1)}(s)\big), \tag{4}$$

where $x_{m,H}^{(k)}(s)$ is the end-effector and object state after H steps in the m-th rollout and d is a distance on that state. Running the rollouts to the end of the episode ($H = T - t$) and comparing returns instead of states gives a direct Monte Carlo estimate of the drop in expected return, $\hat{\Delta}_k^{R}(s) = \frac{1}{M}\sum_m \big(R(\tau_m^{(1)}) - R(\tau_m^{(k)})\big)$, at the highest cost. These estimators require resetting the simulator to arbitrary states, which LIBERO supports.

2.2 Learning the Router

We train the router offline by supervised learning, using the estimated precision gaps as labels (Algorithm 1). Each iteration has a data generation phase, which rolls out a policy and labels the states it visits, and a learning phase, which fits the router to all labeled states so far. The first iteration rolls out π_1; later iterations roll out the routed policy π_ϕ and aggregate its states into the dataset (DAgger-style). This matters because routing decisions change which states are reached, so the router must be trained on the states it will actually encounter.

Algorithm 1 Offline router learning

Require: Quantized policies $\{\pi_k\}_{k=1}^K$ with costs $\{c_k\}_{k=1}^K$, simulator, gap estimator $\hat{\Delta}$, episodes per iteration N, iterations I

$\mathcal{D} \leftarrow \emptyset$
for $i = 0, 1, \ldots, I$ **do**
  $\pi_{\text{roll}} \leftarrow \pi_1$ **if** $i = 0$ **else** π_ϕ
  Data generation phase
  for $n = 1, \ldots, N$ **do**
    Roll out π_{roll} and record visited states $s_0, \ldots, s_{T-1}$
    for each visited state s_t **do**
      Run all K policies on s_t and compute $\{\hat{\Delta}_k(s_t)\}_{k=1}^K$
      Compute label $y(s_t)$ and weight $w(s_t)$ (Eqs. (5)–(6))
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(z_t, y(s_t), w(s_t))\}$
    end for
  end for
  Learning phase
  Train g_ϕ on $\mathcal{D}$ by minimizing $\mathcal{L}_{\text{CE}}(\phi)$ (Eq. (7))
end for
return g_ϕ

Learning loss. Each state receives a hard label, either the precision with the best tradeoff between gap and cost, or the lowest precision whose gap is within a tolerance ϵ:

$$y(s) = \arg\min_k \left[\hat{\Delta}_k(s) + \lambda c_k\right] \qquad \text{or} \qquad y(s) = \max\{k : \hat{\Delta}_k(s) \le \epsilon\}. \tag{5}$$

Misrouting is most costly at critical states, where the low-precision policies diverge most from π_1. We therefore weight each state by

$$w(s) = 1 + \alpha \max_k \hat{\Delta}_k(s), \tag{6}$$

and train the router with a weighted cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(\phi) = -\,\mathbb{E}_{(z,y,w)\sim\mathcal{D}}\big[\, w \,\log g_\phi(y \mid z)\,\big]. \tag{7}$$

3 Experiments

4 Evaluation

We will evaluate the resulting system on:

- task success rate,
- inference latency,
- energy usage, and
- roofline analysis.